

Estudio Java, JavaFx y patrón MVC

María Camila Valbuena Diaz

Diego Alejandro Gaitán Velandia

Laura Valentina Castillo Linares

Practica a Aplicada Sistemas

Docente: Soto Forero Wilson Eduardo

Politécnico GranColombiano

Bogotá - Colombia

05/02/2026

Java (POO, Clases y objetos)

¿Qué es la programación orientada a objetos?

es un paradigma de programación que organiza el software en torno a **objetos**, los cuales representan entidades del mundo real. Cada objeto combina **atributos** (datos) y **métodos** (comportamientos), lo que facilita el diseño de programas más estructurados, reutilizables y fáciles de mantener.

¿Que son las clases en java?

Se puede definir como un molde o plantilla a partir del cual se crean los objetos. Una clase describe las características (atributos) y acciones (métodos) que tendrán los objetos que se instancien a partir de ella.

Las clases están compuestas principalmente por:

- Atributos: son las variables que representan el estado del objeto, como nombre, edad o precio.
- Métodos: son las funciones que definen el comportamiento del objeto, es decir, lo que el objeto puede hacer.
- Constructor: es un método especial que se ejecuta al crear un objeto y sirve para inicializar sus atributos.
- Modificadores de acceso: controlan la visibilidad de los atributos y métodos (public, private, protected).

¿que son los objetos en java?

En **Java**, un **objeto** es una **instancia de una clase**. Representa una entidad concreta que se crea a partir de una clase y que puede interactuar con otros objetos dentro de un programa. Mientras la clase define la estructura y el comportamiento, el objeto es la realización real de esa definición.

Cada objeto tiene un estado, representado por sus atributos los cuales almacenan datos específicos. Comportamiento el cual está definido por sus métodos, que indican las acciones que el objeto puede realizar. Id lo cual lo diferencia de otros objetos, aunque puedan tener los mismos valores.

En Java, los objetos se crean usando la palabra reservada **new**, lo que permite reservar espacio en memoria y ejecutar el constructor de la clase. Cada objeto es independiente, por lo que modificar uno no afecta a otros creados a partir de la misma clase.

Los objetos son esenciales en la **Programación Orientada a Objetos (POO)**, ya que permiten modelar elementos del mundo real, facilitar la reutilización del código y mejorar la

organización del software. A través de la interacción entre objetos se construyen aplicaciones más flexibles, escalables y fáciles de mantener.

¿Qué es un método en POO?

Un método es una acción o comportamiento que pertenece a una clase dentro de la Programación Orientada a Objetos. Define lo que un objeto puede hacer. Mientras los atributos representan características, los métodos representan acciones.

Elementos que componen un método

Todo método tiene las siguientes partes fundamentales:

- Modificador de acceso: define quién puede usar el método (público, privado, protegido).
- Tipo de retorno: indica qué tipo de dato devuelve el método.
- Nombre del método: identifica la acción que realiza.
- Parámetros: datos que el método puede recibir para funcionar.
- Cuerpo del método: conjunto de instrucciones que ejecuta.

Tipos de métodos

- **Métodos sin retorno**

Son métodos que ejecutan una acción, pero no devuelven ningún valor. Se utilizan cuando solo se necesita realizar una tarea.

- **Métodos con retorno**

Son métodos que realizan un proceso y devuelven un resultado. El tipo de retorno debe coincidir con el tipo de dato que se devuelve.

- **Métodos con parámetros**

Son métodos que reciben información externa para poder trabajar. Los parámetros permiten que el método sea más dinámico y reutilizable.

Métodos de instancia y métodos estáticos

- **Métodos de instancia**

Pertenecen a los objetos creados a partir de una clase. Para utilizarlos es necesario crear primero un objeto.

- **Métodos estáticos**

Pertenecen directamente a la clase y no requieren la creación de un objeto para ser utilizados. Se usan generalmente cuando la acción no depende de atributos específicos de un objeto.

- **Método constructor**

Es un método especial que se ejecuta automáticamente cuando se crea un objeto. Su función principal es inicializar los atributos del objeto.

- **Sobrecarga de métodos**

La sobrecarga ocurre cuando una clase tiene varios métodos con el mismo nombre, pero con diferentes parámetros. Esto permite realizar acciones similares de distintas maneras

¿Qué es JavaFX?

JavaFX es una plataforma y conjunto de librerías de Java utilizada para crear interfaces gráficas de usuario (GUI) modernas, interactivas y multiplataforma. Permite desarrollar aplicaciones de escritorio con diseños más atractivos y dinámicos que los de las interfaces tradicionales.

Entre sus principales características se encuentran:

- Interfaz gráfica moderna: soporta botones, tablas, menús, formularios y componentes avanzados.
- Separación de diseño y lógica: usa archivos FXML para definir la interfaz y Java para la lógica del programa.
- Estilos con CSS: permite personalizar la apariencia de la aplicación de forma similar al desarrollo web.
- Multiplataforma: funciona en Windows, Linux y macOS.
- Integración con POO: se adapta perfectamente al uso de clases, objetos y patrones como MVC.

Con lo anterior JavaFX tiene una estructura organizada que separa claramente responsabilidades. Esto es clave porque aplica principios de POO y el patrón MVC.

1. Clase Principal (Application)

Tiene una estructura organizada que permite separar claramente las responsabilidades dentro de una aplicación. Toda aplicación JavaFX comienza con una clase principal que hereda de la clase Application. Esta clase es el punto de entrada del programa y es la encargada de iniciar la interfaz gráfica. Cuando la aplicación se ejecuta, se crea una ventana principal que en JavaFX se llama Stage.

2. Stage

El Stage representa la ventana como tal, es decir, el espacio donde el usuario ve la aplicación. Dentro de esa ventana se coloca una Scene.

3. Scene

La Scene representa el contenido que se muestra dentro del Stage. Podría decirse que, si el Stage es la ventana, la Scene es todo lo que se ve dentro de ella. La Scene contiene un conjunto de elementos visuales organizados en forma de árbol.

4. Nodo

En JavaFX todo elemento visual es un Node, es decir, un nodo. Los botones, textos, imágenes, tablas y contenedores son nodos. Estos nodos se organizan jerárquicamente, lo que significa que unos pueden contener a otros formando una estructura tipo árbol.

5. Layouts

Para organizar los elementos dentro de la Scene se utilizan contenedores llamados layouts. Los layouts permiten distribuir los componentes en posiciones específicas, ya sea de manera vertical, horizontal, en forma de cuadrícula o divididos por regiones. Estos layouts también son nodos, por lo que forman parte del mismo árbol estructural de la interfaz.

6. FXML (Separación- Diseño- Lógica)

Una característica importante de la estructura de JavaFX es la separación entre diseño y lógica. El diseño de la interfaz puede definirse en archivos FXML, mientras que la lógica del programa se desarrolla en clases Java llamadas controladores o controllers.

7. Controller

El controlador se encarga de manejar los eventos generados por el usuario, como hacer clic en un botón o escribir en un campo de texto. De esta manera, la vista se encarga de mostrar la interfaz, el controlador gestiona la interacción y el modelo contiene los datos y la lógica del negocio, siguiendo el patrón MVC.

8. Ciclo de vida de una aplicación en javaFX

El funcionamiento de una aplicación JavaFX sigue un ciclo claro: primero se inicia la clase principal, luego se crea el Stage, se carga la Scene, se construye el árbol de nodos y finalmente el usuario interactúa con la interfaz. Cada acción del usuario genera un evento que es capturado y gestionado por el controlador.

PATRON MVC

El patrón MVC (Modelo-Vista-Controlador) es una arquitectura que divide una aplicación en tres partes para separar la lógica del programa de la interfaz gráfica y de la interacción del usuario, logrando un sistema más organizado y fácil de mantener.

El Modelo gestiona los datos y la lógica del negocio, sin depender de cómo se muestran. La Vista se encarga de la parte visual y de mostrar la información al usuario sin incluir lógica

compleja. El Controlador actúa como intermediario, recibe las acciones del usuario, se comunica con el modelo y actualiza la vista.

Su funcionamiento sigue un flujo: el usuario interactúa con la vista, el controlador procesa la acción, el modelo realiza los cambios y la vista se actualiza. La principal ventaja de MVC es que facilita el mantenimiento, la escalabilidad y el trabajo en equipo al mantener separadas las responsabilidades.

EJEMPLO BASICO

- Modelo: Se encarga de los datos y la lógica.

```
public class PromedioModel {  
  
    private double nota1;  
    private double nota2;  
  
    public PromedioModel(double nota1, double nota2) {  
        this.nota1 = nota1;  
        this.nota2 = nota2;  
    }  
  
    public double calcularPromedio() {  
        return (nota1 + nota2) / 2;  
    }  
}
```

- Vista

```
public class PersonaVista {  
  
    public void mostrarPersona(String nombre, int edad) {  
        System.out.println("Nombre: " + nombre);  
        System.out.println("Edad: " + edad);  
    }  
}
```

- Controlador

```

public class PersonaControlador {

    private Persona modelo;
    private PersonaVista vista;

    public PersonaControlador(Persona modelo, PersonaVista vista) {
        this.modelo = modelo;
        this.vista = vista;
    }

    public void setNombre(String nombre) {
        modelo.setNombre(nombre);
    }

    public String getNombre() {
        return modelo.getNombre();
    }

    public void setEdad(int edad) {
        modelo.setEdad(edad);
    }

    public int getEdad() {
        return modelo.getEdad();
    }

    public void actualizarVista() {
        vista.mostrarPersona(modelo.getNombre(), modelo.getEdad());
    }

}

```

REFERENCIAS

- *JavaFX Overview (Release 8)*. (2025, 20 octubre). <https://docs.oracle.com/javase/8/javafx/get-started-tutorial/jfx-overview.htm>
- GeeksforGeeks. (2026, 23 enero). *Java OOP(Object Oriented Programming) concepts*. GeeksforGeeks. <https://www.geeksforgeeks.org/java/object-oriented-programming-oops-concept-in-java/>

